

Criação da mecânica da Fase 4

Disciplina: Certificadora de Competência Comum - 2026/01

Grupo 1: Bruna Kaori Takuti, Fernando Souza, Danilo Cândido, Giovana Furlan e Giovana Ribeiro.

Interface:

- Resumo da matéria sobre técnicas de modularização e funções em C
- Vídeo-aulas disponíveis sobre o conteúdo
- O usuário terá acesso às quests (desafios) organizadas por fase
- Cada desafio apresentará um enunciado com o objetivo da atividade
- O usuário deverá escrever o código no terminal proposto
- Após a execução, o sistema irá validar se o código está correto

Caso o código funcione corretamente o usuário poderá avançar para a próxima fase, caso contrário o sistema retornará um erro.

Tópicos abordados:

- Declaração e chamada de funções
- Funções `void` (sem retorno)
- Parâmetros (passagem de dados)
- `return` (valor de retorno)
- Modularização em arquivos `.h` (cabeçalho) e `.c` (implementação)

Objetivo: compreender e aplicar técnicas de **modularização** em C, criando **funções** reutilizáveis com parâmetros e retorno, separando o código em módulos para promover legibilidade, reuso (DRY) e trabalho em equipe.

Jogo da Fase 4: ARSENAL DO HACKER

Nesta fase, o jogador deverá montar um **arsenal de ferramentas** (funções) que o hacker utilizará para executar seus ataques. Em vez de escrever todo o código dentro do `main()`, o jogador aprenderá a **dividir o programa em funções** especializadas — cada uma fazendo uma única tarefa — e a chamá-las quando precisar.

Quest 1 - Criar uma função `void` que exiba o banner do hacker na tela

O jogador deverá criar uma função sem retorno (`void`) que apenas execute uma ação: imprimir o banner de boas-vindas do sistema invadido. Em seguida, deverá chamá-la dentro do `main()`.

Dica: funções `void` não usam `return`, apenas executam uma tarefa.

Exemplo de código:

```
void exibirBanner() {
    printf("=====\n");
    printf("  ACESSO HACKER LIBERADO\n");
    printf("=====\n");
}

int main() {
    exibirBanner();
    return 0;
}
```

Quest 2 - Criar uma função com parâmetros para registrar o alvo do ataque

O jogador deverá criar uma função que receba o **nome do alvo** como parâmetro e imprima uma mensagem personalizada. O objetivo é entender como enviar dados ("ingredientes") para dentro de uma função.

Dica: os parâmetros vão entre os parênteses da função, separados por vírgula.

Exemplo de código:

```
void registrarAlvo(char nome[]) {
    printf("Alvo identificado: %s\n", nome);
}

int main() {
    registrarAlvo("Servidor Central");
    return 0;
}
```

Quest 3 - Criar uma função com retorno (`int`) para calcular o dano do ataque

O jogador deverá criar uma função que receba dois valores inteiros (força do hacker e bônus da ferramenta) e retorne a soma deles, representando o dano causado. O valor retornado deve ser **armazenado em uma variável** no `main()` e impresso na tela.

Dica: o tipo de retorno fica antes do nome da função (ex: `int`). Use `return` para devolver o resultado.

Exemplo de código:

```
int calcularDano(int forca, int bonus) {  
    return forca + bonus;  
}  
  
int main() {  
    int dano = calcularDano(50, 30);  
    printf("Dano total causado: %d\n", dano);  
    return 0;  
}
```

Quest 4 - Combinar várias funções no `main()` para executar um ataque completo

O jogador deverá utilizar as três funções criadas anteriormente (`exibirBanner` , `registrarAlvo` e `calcularDano`) **chamando-as em sequência** dentro do `main()` . O objetivo é mostrar como a modularização deixa o programa principal limpo e legível, igual a um livro.

Dica: o `main()` deve ficar curto — ele apenas **orquestra** as chamadas; toda a lógica fica dentro das funções.

Exemplo de código:

```
int main() {  
    exibirBanner();  
    registrarAlvo("Servidor Central");  
  
    int dano = calcularDano(50, 30);  
    printf("Dano total causado: %d\n", dano);  
  
    return 0;  
}
```

Quest 5 - Separar o arsenal em arquivos `.h` e `.c` (modularização de verdade)

O jogador deverá organizar suas funções em arquivos separados, simulando como um time de desenvolvimento divide o trabalho. As **assinaturas** das funções ficam no arquivo de cabeçalho (`.h`) e a **implementação** fica no arquivo `.c`. O `main.c` apenas inclui o cabeçalho e usa as funções.

Dica: use `#include "arsenal.h"` para importar as funções declaradas no cabeçalho.

Exemplo de código:

arsenal.h (assinaturas — o "menu" do que o módulo sabe fazer):

```
void exibirBanner();
void registrarAlvo(char nome[]);
int calcularDano(int forca, int bonus);
```

arsenal.c (implementação — o "código real"):

```
#include <stdio.h>
#include "arsenal.h"

void exibirBanner() {
    printf("=====\n");
    printf("  ACESSO HACKER LIBERADO\n");
    printf("=====\n");
}

void registrarAlvo(char nome[]) {
    printf("Alvo identificado: %s\n", nome);
}

int calcularDano(int forca, int bonus) {
    return forca + bonus;
}
```

main.c (programa principal):

```
#include <stdio.h>
#include "arsenal.h"

int main() {
    exibirBanner();
    registrarAlvo("Servidor Central");

    int dano = calcularDano(50, 30);
    printf("Dano total causado: %d\n", dano);

    return 0;
}
```

Vantagens demonstradas na fase

- **Reutilização (DRY):** escreva a função uma vez, chame quantas vezes quiser.
- **Trabalho em equipe:** cada membro pode implementar um módulo diferente.
- **Legibilidade e debug:** se o cálculo do dano falhar, o jogador já sabe exatamente em qual função procurar o erro.